Regex Generator from Natural Language

Technical Requirements Document (TRD)

Version 0.2 — Updated 16 October 2026

Project status: PLANNING ONLY — coding has not started

1. Purpose and Status

This document proposes a technical design for the planned application. It distinguishes confirmed choices from suggestions. The project is in planning only; no implementation is claimed.

2. Confirmed vs. Unconfirmed Decisions

Area	Decision	Status
Frontend	HTML, CSS, JavaScript	Confirmed
Model access	Cloud API model	Confirmed approach
Model vendor / model ID	Not selected	Undecided
Backend framework	Not selected; server-side API layer is recommended	Undecided
Regex engine	Not selected; choose and document a supported flavor	Undecided
Data storage	No database decision; MVP may not need persistent storage	Undecided
Deployment platform	Not selected	Undecided
Authentication	Not decided; likely unnecessary for a basic no-account MVP	Undecided

3. Proposed Architecture

Recommended request path: Browser UI → application backend → cloud model API. The backend parses and constrains model output, validates the Regex with the chosen engine, and returns a structured response. The browser should never call a secret-bearing model API directly.

Component	Responsibility	Decision status
Frontend: HTML/CSS/JavaScript	Requirement form, output panel, explanation, sample-string tester, status and error display.	Confirmed technology; UI structure proposed
Backend API	Input validation, prompt assembly, model API call, output parsing, Regex validation, rate limiting.	Required design role; framework undecided
Cloud model API	Interpret natural language and propose a Regex plus explanation/assumptions.	Cloud approach confirmed; provider/model undecided
Regex engine	Compile/check syntax and execute sample-string matching.	Required capability; implementation/flavor undecided
Optional persistence	Store history or saved patterns only if later approved.	Not in MVP by default; undecided

4. Proposed Technology Options (Not Final Selections)

Layer	Possible options	Status / selection criteria
Backend	Python with Flask or FastAPI; alternatively another suitable server framework	Not selected; compare familiarity, deployment, validation and testing needs
Request/response schema	JSON; optional schema validation library such as Pydantic if Python is	Proposed

	chosen	
Regex validation	Python `re` is one possible initial engine if Python is selected	Example only; flavor compatibility must be assessed
Model integration	Provider's official SDK or HTTPS API from backend	Provider undecided
Testing	Unit tests plus API integration tests and a curated Regex corpus	Approach proposed; framework undecided
Version control	Git and GitHub	Suggested, not confirmed

5. API Contract (Proposed)

Endpoint examples below are proposed contracts, not implemented endpoints.

POST /api/regex/generate

Request example:

```
{
  "requirement": "Match a 5-digit postal code",
  "flavor": "default",
  "options": {
    "caseInsensitive": false,
    "explain": true
  }
}
```

The `flavor` value is a placeholder until supported flavors are selected. The API should reject unsupported values rather than silently changing behavior.

Proposed response fields:

- `pattern`: generated Regex string.
- `flavor`: actual Regex engine/flavor used.
- `valid`: whether deterministic validation passed.
- `explanation`: plain-language explanation, preferably verified against the final pattern.
- `constraints`: extracted requirements and assumptions.
- `warnings`: limitations, ambiguities, or security caveats.
- `requestId`: optional non-sensitive correlation identifier.

POST /api/regex/test

Proposed request: `{ "pattern": "...", "flavor": "...", "testStrings": ["...", "..."] }`.

Proposed response: validation status and per-string match results, including match spans only if supported. Apply request-size, string-count, and execution-time limits.

Error format (proposed)

```
{
  "error": {
    "code": "INVALID_REGEX",
    "message": "The pattern could not be compiled by the selected engine."
  }
}
```

6. Model Output Handling

1. Send the user requirement with explicit instructions to return a constrained structured response.
2. Parse the response as data; do not trust free-form text as executable instructions.
3. Reject missing or wrongly typed required fields.
4. Validate the candidate Regex with the selected engine.
5. Generate or verify the explanation against the actual pattern where feasible.
6. If validation fails, allow a bounded correction attempt or return a clear failure; avoid unbounded retries.
7. Keep user input and model output out of logs unless necessary and appropriately redacted.

7. Security, Privacy, and Abuse Controls

- Store cloud API credentials in server-side environment configuration or a secrets manager; never ship them in frontend assets.
- Validate request body size, text length, test-string count, and allowed options.
- Use HTTPS in deployment and apply rate limits to protect provider quotas and costs.
- Treat generated patterns as untrusted. Avoid evaluating generated code or embedding model output as HTML.
- Consider catastrophic backtracking and ReDoS. Use execution timeouts or a safer engine where available; do not assume every engine supports safe timeouts.
- Do not persist prompts or test strings by default unless a later product decision requires it.
- Return generic public errors while keeping sensitive diagnostics server-side.

8. Reliability and Observability

- Configure request timeouts and bounded retries for transient provider errors.
- Handle quota exhaustion, provider outage, malformed model output, and invalid Regex separately.
- Track latency, success/error counts, validation failures, and approximate cost without logging secrets or raw sensitive inputs.
- Consider a provider abstraction only if a second provider is actually needed; multi-provider fallback is not currently confirmed.

9. Testing Strategy

Test group	What to test
Unit	Input validation, response parsing, pattern validation, error mapping.
Functional	Natural-language request to pattern; explanation and sample testing.
Regression corpus	Common formats, boundaries, optional parts, anchors, escaping, Unicode, positive and negative examples.
Ambiguity	Requirements such as “valid username” that need clarification.
Failure handling	Timeouts, quota errors, provider 5xx, malformed responses, invalid Regex.
Security	Oversized inputs, HTML/script injection in displayed text, rate limits, ReDoS cases.
Browser / accessibility	Keyboard-only use, labels, responsive layout, readable validation states.

10. Deployment Plan (To Be Finalized)

- Choose hosting only after backend and model provider are selected.
- Configure environment-specific secrets outside source control.
- Set usage budgets, quotas, monitoring, and a simple rollback process.
- Provide health checks and safe error responses.
- Document local setup, required environment variables, and deployment steps once stack decisions are final.

11. Open Technical Decisions

- Cloud provider, model name/version, SDK, API pricing and retention terms.
- Backend framework and runtime.
- Regex engine, flavor, Unicode behavior, matching semantics and timeout capabilities.
- Whether the generation and testing endpoints are separate or combined.
- Whether to support streaming responses or keep a simple request/response flow.
- Hosting, monitoring, environment management, and rate-limit policy.
- Automated test framework and target coverage.

Document status: technical proposal only. All options listed as proposed are not final decisions, and no implementation is claimed.